

RPG Project — Design Note

Phần 1: Thiết kế • Final v2

Mục tiêu project

- Học OOP: kế thừa, phân chia trách nhiệm giữa các class
- Chiến đấu theo lượt, người chơi input
- Không dùng AP — chỉ dùng cooldown để giới hạn skill
- Win condition: hết spawn pool VÀ hết quân trên sân

Thông số nhân vật

Chỉ số cơ bản

Thông số	Warrior	Mage
max_hp	120	75
atk cơ bản	20	18
rest hồi máu / lượt	15	12

Skill & Cooldown

Skill	Damage / Effect	Cooldown
Tấn công thường	atk — mất lượt	0
[Warrior] critical_strike	atk x 2.0 — đơn mục tiêu	3 lượt
[Warrior] power_buff	atk +15 trong 2 lượt, không mất lượt	2 lượt
[Mage] aoe_attack	atk x 1.0 lên tối đa 2 mục tiêu	3 lượt
[Mage] heal_ally	Hồi 35 HP cho 1 đồng minh, không mất lượt	2 lượt

Hệ số đặc biệt

Quy tắc	Hệ số
---------	-------

Warrior hết buff → debuff atk	-10 trong 1 lượt
Bật lại buff khi đang debuff	Xóa debuff, không cộng dồn
Mage đánh Mage địch khi còn Warrior địch	damage x 0.4

Class Structure

Class Base (không triệu hồi trực tiếp)

Attribute	Ghi chú
name	Tên nhân vật
hp	Máu hiện tại
max_hp	Máu tối đa
atk	Sát thương cơ bản
cooldowns	Dict lưu cooldown từng skill
buff_turns_left	Số lượt buff còn lại
is_debuffed	Đang bị debuff hay không
is_alive	Còn sống hay không

Method	Ghi chú
attack(target)	Tấn công thường
take_damage(amount)	Nhận sát thương, gọi die() nếu $HP \leq 0$
die()	Đánh dấu is_alive = False
rest()	Hồi máu, mất lượt
use_skill(skill_name, target)	Dùng skill
update_status()	Tự cập nhật buff/debuff/cooldown cuối lượt

Class Warrior (kế thừa Base)

- max_hp = 120, atk = 20
- Skill: critical_strike, power_buff

- Giới hạn: không đánh Mage địch nếu còn Warrior địch (BM kiểm tra)

Class Mage (kế thừa Base)

- max_hp = 75, atk = 18
- Skill: aoe_attack, heal_ally
- heal_ally ưu tiên: ít máu nhất → nếu bằng HP thì ưu tiên Warrior
- Giới hạn: damage lên Mage địch x 0.4 nếu còn Warrior địch (BM kiểm tra)

Class BattleManager

Attribute	Ghi chú
team_a	Danh sách quân team A trên sân
team_b	Danh sách quân team B trên sân
spawn_pool_a	Số lượt spawn còn lại team A (bắt đầu: 3)
spawn_pool_b	Số lượt spawn còn lại team B (bắt đầu: 3)
turn_count	Tổng số lượt đã qua
current_turn	'A' hoặc 'B'

Method	Ghi chú
setup_phase()	Mỗi bên spawn 2 quân luân phiên, không +turn
spawn(team, char_class)	Tạo object + thêm vào team + trừ spawn pool
start_turn()	Thông báo bắt đầu lượt
get_input()	Nhận input + validate + kiểm tra cooldown
apply_action()	Thực thi hành động, tính damage, gọi die() nếu cần
end_turn()	Gọi update_status() cho tất cả, +turn, chuyển lượt
check_win()	spawn pool = 0 VÀ hết quân trên sân → thua

Workflow

Setup Phase

A spawn → B spawn → A spawn → B spawn

(Không +turn | Trừ spawn pool | Mỗi bên dùng 2/5 lượt spawn)

Battle Phase (loop)

start_turn()

→ get_input() # spawn / chọn nhân vật + hành động + mục tiêu

→ apply_action() # thực thi, tính damage, die() nếu $HP \leq 0$

→ end_turn() # update_status() toàn sân + chuyển lượt

→ check_win() # spawn pool = 0 VÀ hết quân → thua

Quy tắc BM kiểm tra

- Warrior không đánh Mage địch nếu còn Warrior địch
- Mage damage lên Mage địch x 0.4 nếu còn Warrior địch
- Spawn tốn 1 lượt trong battle phase
- Không có bỏ lượt — chỉ có nhận thua
- Spawn pool mỗi bên: 5 tổng (2 setup + 3 battle)
- Số quân trên sân: không giới hạn

❑ Version 2 (sau khi V1 chạy được)

- Thêm class Base (công trình) có HP
- Warrior/Mage không đánh Base khi còn quân địch
- Mage AOE đánh được Base khi quân địch < 2
- Win condition mới: phá Base địch
- Hòa: cả 2 Base còn máu nhưng hết quân

RPG Project — Hướng dẫn Code

Phần 2: Gợi ý từng bước • Dành cho người mới học OOP

Mỗi bước bên dưới gợi ý cấu trúc và những gì cần nghĩ tới — không đưa code sẵn. Làm xong một bước, test chạy được rồi mới qua bước tiếp.

Bước 1 — Class Base

Đây là nền tảng của toàn bộ project. Viết xong bước này là bạn có một nhân vật 'trần' hoạt động được.

Cần làm gì?

Việc	Gợi ý nghĩ
Viết <code>__init__</code>	Nhận name, hp, atk làm tham số. <code>max_hp = hp</code> lúc khởi tạo.
<code>take_damage(amount)</code>	Trừ HP. Nếu $HP \leq 0 \rightarrow$ gọi <code>die()</code> . Không để HP âm.
<code>die()</code>	Chỉ cần đặt <code>is_alive = False</code> và in thông báo.
<code>rest()</code>	Cộng HP nhưng không vượt <code>max_hp</code> — dùng <code>min()</code> để giới hạn.
<code>attack(target)</code>	Gọi <code>target.take_damage(self.atk)</code> . Target là object khác.
<code>__str__</code>	In tên + HP hiện tại. Hữu ích khi debug.

>> Test ngay sau bước này: tạo 2 object Character, cho cái này attack cái kia, kiểm tra HP giảm đúng không.

Bước 2 — Class Warrior và Mage

Đây là lúc kế thừa (inheritance) xuất hiện. Warrior và Mage kế thừa toàn bộ từ Base — chỉ cần ghi đè những gì khác.

Cần làm gì?

Việc	Gợi ý nghĩ
<code>class Warrior(Base)</code>	Dùng <code>super().__init__()</code> để gọi <code>__init__</code> của Base.

Override <code>__init__</code>	Chỉ cần đổi giá trị <code>max_hp</code> và <code>atk</code> — không viết lại từ đầu.
Chưa cần skill	Để trống phần skill, sẽ thêm ở Bước 4.
Tương tự cho Mage	Giống Warrior — chỉ khác số liệu <code>max_hp</code> và <code>atk</code> .

>> Test: tạo Warrior và Mage, gọi `attack()` qua lại. Lúc này chúng dùng `attack()` kế thừa từ Base — không cần viết lại.

Bước 3 — BattleManager cơ bản

Mục tiêu của bước này: có một vòng lặp chiến đấu chạy được, dù chưa có skill hay luật đặc biệt gì.

Cần làm gì?

Việc	Gợi ý nghĩ
<code>__init__</code>	Khởi tạo <code>team_a=[]</code> , <code>team_b=[]</code> , <code>spawn_pool_a=3</code> , <code>spawn_pool_b=3</code> , <code>turn_count=0</code> , <code>current_turn='A'</code> .
<code>setup_phase()</code>	Vòng lặp 4 lần: A spawn → B spawn → A spawn → B spawn. Mỗi lần hỏi người chơi muốn spawn Warrior hay Mage.
<code>spawn(team, cls)</code>	Hỏi tên nhân vật, tạo object (Warrior hoặc Mage), append vào team, trừ <code>spawn_pool</code> .
<code>start_turn()</code>	In 'Luợt của team A/B'. In danh sách quân 2 bên còn sống.
<code>get_input()</code>	Hỏi: spawn hay chọn nhân vật? Nếu chọn nhân vật → hỏi hành động: tấn công / nghỉ.
<code>apply_action()</code>	Dựa vào lựa chọn → gọi method tương ứng của nhân vật.
<code>end_turn()</code>	<code>+turn_count</code> , đổi <code>current_turn</code> .
<code>check_win()</code>	Lọc danh sách quân còn <code>is_alive</code> . Nếu <code>spawn_pool = 0</code> và list rỗng → thua.

>> Chỉ cần tấn công thường và nghỉ — không cần skill. Mục tiêu là vòng lặp chạy từ đầu đến cuối, win condition hoạt động.

Bước 4 — Skill và Cooldown

Thêm skill vào từng nhân vật. Đây là bước thêm state management — nhân vật tự quản lý trạng thái của mình.

Cần làm gì?

Việc	Gợi ý nghĩ
Thêm cooldowns vào Base.__init__	cooldowns = {'skill1': 0, 'skill2': 0}. Giá trị 0 = sẵn sàng dùng.
update_status() trong Base	Giảm tất cả cooldown > 0 xuống 1. Xử lý buff_turns_left và is_debuffed.
BM gọi update_status()	Trong end_turn(): duyệt tất cả quân 2 bên còn sống → gọi update_status().
Warrior.critical_strike(target)	Nếu cooldowns['critical'] > 0 → thông báo còn cooldown. Ngược lại → damage x2, đặt cooldown = 3.
Warrior.power_buff()	Tăng atk +15, đặt buff_turns_left = 2, cooldown = 2.
Xử lý buff trong update_status()	Nếu buff_turns_left > 0 → giảm 1. Nếu về 0 → is_debuffed = True, atk -= 10.
Mage.aoe_attack(targets)	targets là list tối đa 2 nhân vật. Gọi take_damage() cho từng cái.
Mage.heal_ally(target)	target.hp += 35, không vượt max_hp — dùng min().

>> Thêm từng skill một, test xong mới thêm skill tiếp theo. Đừng thêm hết một lúc rồi debug cùng lúc.

Bước 5 — Luật đặc biệt trong BattleManager

Bước cuối: thêm các luật liên quan đến toàn bộ sân đấu. Những luật này nằm trong BM vì chúng cần nhìn thấy cả 2 team.

Cần làm gì?

Luật	Gợi ý làm
Warrior không đánh Mage địch nếu còn Warrior địch	Trong get_input() hoặc apply_action(): kiểm tra enemy_team có Warrior còn sống không trước khi cho chọn Mage làm mục tiêu.

Mage damage x 0.4 lên Mage địch nếu còn Warrior địch	Trong <code>apply_action()</code> : nếu attacker là Mage và target là Mage và <code>enemy_team</code> còn Warrior → nhân damage với 0.4 trước khi gọi <code>take_damage()</code> .
Validate target hợp lệ	Trong <code>get_input()</code> : chỉ hiện những mục tiêu <code>is_alive = True</code> cho người chơi chọn.

>> Sau bước này project hoàn chỉnh. Nếu muốn thêm Version 2 (Base có HP, win condition mới) thì đây là lúc — không cần sửa code cũ, chỉ thêm class Base mới và mở rộng BM.

□ Tóm tắt thứ tự làm

Bước	Làm gì	Biết mình xong khi nào
1	Class Base	2 object đánh nhau, HP giảm đúng, <code>die()</code> hoạt động
2	Warrior + Mage kế thừa	Tạo được Warrior/Mage, dùng được <code>attack()</code> từ Base
3	BattleManager vòng lặp cơ bản	Chơi được 1 ván hoàn chỉnh, chỉ có tấn công + nghỉ
4	Skill + cooldown	Dùng được skill, cooldown đếm ngược đúng
5	Luật đặc biệt trong BM	Warrior không đánh nhầm, Mage bị giảm damage đúng